Securitize EVM Whitelister Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Dacian

Stalin

February 11, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**
- 4.0.1 Vault Whitelister . . . 2
- 4.0.2 Uniswap V4 AllowListChecker . . . 2

5 Audit Scope **2**

6 Executive Summary **3**

7 Findings **5**
- 7.1 Informational . . . 5
  - 7.1.1 Incorrect pragma as support for defining operators on user-defined value types was added in Solidity 0.8.19 . . . 5
  - 7.1.2 `PermissionFlags::hasFlag` can't be used to check for `PermissionFlag::NONE` . . . 5
  - 7.1.3 `PermissionFlags::hasFlag` incorrectly grants `ALL_PERMISSIONS` to `PermissionFlag::LIQUIDITY_ALLOWED` and `PermissionFlag::SWAP_ALLOWED` . . . 6
  - 7.1.4 `PermissionFlags::hasFlag` inconsistent validation for combined flags . . . 7
  - 7.1.5 `PermissionFlags::and` function will encode the permission for `SWAP_ALLOWED` AND `LIQUIDITY_ALLOWED` as if the account would have `NONE` permissions . . . 8
  - 7.1.6 Implement recommended changes to proposed vault whitelister standard . . . 8
- 7.2 Gas Optimization . . . 9
  - 7.2.1 `BaseWhitelister::addOperator, removeOperator` should use `_grantRole` and `_revokeRole` directly and only emit events when they return `true` . . . 9
  - 7.2.2 `Whitelister::whitelist` duplicates checks already performed in `RegistryService::addWallet` . . . 9

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Securitize is adding two new components enforcing permissioning based on Securitize DS Token whitelisting.

4.0.1 Vault Whitelister

The Vault `Whitelister` allows an external protocol to whitelist a vault address they deploy under the identity of an investor, based on the investor's status in Securitize Registry Service. This enables vaults to operate on behalf of investors while preserving DS Token's compliance model.

4.0.2 Uniswap V4 AllowListChecker

The `AllowListChecker` is designed for permissioned Uniswap v4 pools, determining whether a given address is allowed to participate in swaps and add/remove liquidity. Permission decisions are based on whether a given input address is registered to an investor via the Securitize Registry Service.

5 Audit Scope

The audit scope was limited to:

```
1-vault-whitelister/contracts/BaseWhitelister.sol
1-vault-whitelister/contracts/Whitelister.sol
2-allowlist-checker/contracts/AllowListChecker.sol
2-allowlist-checker/contracts/uniswap/permissionedPools/libraries/PermissionFlags.sol
2-allowlist-checker/contracts/uniswap/permissionedPools/BaseAllowListChecker.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the Securitize EVM Whitelister smart contracts provided by Securitize. In this period, a total of 8 issues were found.

There were no significant findings, only informational and gas optimizations.

Summary

Project Name	Securitize EVM Whitelister
Repository	bc-vault-whitelister
Commit	ff8768600492...
Fix Commit	5ceab6d815cc...
Repository 2	bc-allowlist-checker-sc
Commit	7c3e77f7d01d...
Fix Commit	0b8d5061582f...
Repository 3	dstoken
Commit	8ddd7f374374...
Audit Timeline	Jan 26th - Jan 27th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Informational	6
Gas Optimizations	2
Total Issues	8

Summary of Findings

[I-1] Incorrect pragma as support for defining operators on user-defined value types was added in Solidity 0.8.19	Resolved
[I-2] `PermissionFlags::hasFlag` can't be used to check for `PermissionFlag::NONE`	Resolved
[I-3] `PermissionFlags::hasFlag` incorrectly grants `ALL_PERMISSIONS` to `PermissionFlag::LIQUIDITY_ALLOWED` and `PermissionFlag::SWAP_ALLOWED`	Resolved

[I-4] `PermissionFlags::hasFlag` inconsistent validation for combined flags	Resolved
[I-5] `PermissionFlags::and` function will encode the permission for `SWAP_ALLOWED` AND `LIQUIDITY_ALLOWED` as if the account would have `NONE` permissions	Resolved
[I-6] Implement recommended changes to proposed vault whitelister standard	Resolved
[G-1] `BaseWhitelister::addOperator, removeOperator` should use `_grantRole` and `_revokeRole` directly and only emit events when they return `true`	Resolved
[G-2] `Whitelister::whitelist` duplicates checks already performed in `RegistryService::addWallet`	Acknowledged

7 Findings

7.1 Informational

7.1.1 Incorrect pragma as support for defining operators on user-defined value types was added in Solidity 0.8.19

Description: These files have the following pragma:

- `contracts/uniswap/permissionedPools/libraries/PermissionFlags.sol`
- `contracts/uniswap/permissionedPools/BaseAllowListChecker.sol`

```
pragma solidity ^0.8.0;
```

However this is incorrect as support for defining operators on user-defined value types was added in Solidity 0.8.19.

Recommended Mitigation: Use the correct pragma:

```
- pragma solidity ^0.8.0;
+ pragma solidity ^0.8.19;
```

Securitize: Fixed in commit f938341 by using 0.8.22 consistent with the other contracts.

Cyfrin: Verified.

7.1.2 `PermissionFlags::hasFlag` can't be used to check for `PermissionFlag::NONE`

Description: `PermissionFlags.sol` defines some operator functions, constants and a `hasFlag` function; examining these last two:

```
    PermissionFlag constant NONE = PermissionFlag.wrap(0x0000);
    PermissionFlag constant SWAP_ALLOWED = PermissionFlag.wrap(0x0001);
    PermissionFlag constant LIQUIDITY_ALLOWED = PermissionFlag.wrap(0x0002);
    PermissionFlag constant ALL_ALLOWED = PermissionFlag.wrap(0xFFFF);

    function hasFlag(PermissionFlag permissions, PermissionFlag flag) internal pure returns (bool) {
        return PermissionFlag.unwrap(and(permissions, flag)) != 0;
    }
```

One side-effect of this implementation is that `hasFlag(ANY_VALUE, PermissionFlags.NONE)` always returns `false` since:

```
hasFlag(ANY_VALUE, PermissionFlags.NONE)
// return ANY_VALUE & 0x0000 = 0x0000
// return 0x0000 != 0 → false
```

Recommended Mitigation: * Add a separate function in `PermissionFlags` to check this using direct equality:

```
function hasNoPermissions(PermissionFlag permissions) internal pure returns (bool) {
    return permissions == NONE;
}
```

- Revert in `hasFlag` if the input `flag` is `NONE` to prevent this erroneous use case:

```
+ error InvalidFlagCheck();

function hasFlag(PermissionFlag permissions, PermissionFlag flag) internal pure returns (bool) {
+   if (flag == NONE) revert InvalidFlagCheck();
    return PermissionFlag.unwrap(and(permissions, flag)) != 0;
}
```

Securitize: Remove this functionality in commit 0b8d506 as it was not being used.

Cyfrin: Verified.

7.1.3 `PermissionFlags::hasFlag` incorrectly grants `ALL_PERMISSIONS` to `PermissionFlag::LIQUIDITY_ALLOWED` and `PermissionFlag::SWAP_ALLOWED`

Description: `PermissionFlags::hasFlag` returns true when checking the flag `LIQUIDITY_ALLOWED` against the flag `ALL_ALLOWED`; This means, when checking if an account allowed ONLY to add liquidity has permissions to do any operation (ALL_ALLOWED), `PermissionFlags::hasFlag` would incorrectly signal that the account indeed has permissions to do any operation.

- The same affects the `SWAP_ALLOWED` flag.
- The same affects validating the combination of `SWAP_ALLOWED` or `LIQUIDITY_ALLOWED`, `hasFlag` signals this combination as if it would be the `ALL_ALLOWED` flag.

Impact: Accounts flagged only to add liquidity or do swaps would receive ALL permissions, allowing them to perform operations they should not be allowed to.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import {PermissionFlag, PermissionFlags} from
↪  "../../contracts/uniswap/permissionedPools/libraries/PermissionFlags.sol";

contract PermissionFlagsTest is Test {
    using PermissionFlags for PermissionFlag;

    // Helper to make tests more readable
    function _has(PermissionFlag permissions, PermissionFlag flag) internal pure returns (bool) {
        return permissions.hasFlag(flag);
    }

    function test_hasFlag_SWAP_ALLOWED() public pure {
        PermissionFlag p = PermissionFlags.SWAP_ALLOWED;
        // assertFalse(_has(p, PermissionFlags.NONE));
        // assertTrue(_has(p, PermissionFlags.SWAP_ALLOWED));
        // assertFalse(_has(p, PermissionFlags.LIQUIDITY_ALLOWED));

        //@audit-issue => SWAP_ALLOWED flag is given ALL_ALLOWED permissions!
        assertTrue(_has(p, PermissionFlags.ALL_ALLOWED)); // single flag != ALL
    }

    function test_hasFlag_LIQUIDITY_ALLOWED() public pure {
        PermissionFlag p;

        p = PermissionFlags.LIQUIDITY_ALLOWED;
        // assertFalse(_has(p, PermissionFlags.NONE));
        // assertFalse(_has(p, PermissionFlags.SWAP_ALLOWED));
        // assertTrue(_has(p, PermissionFlags.LIQUIDITY_ALLOWED));

        //@audit-issue => LIQUIDITY_ALLOWED flag is given ALL_ALLOWED permissions!
        assertTrue(_has(p, PermissionFlags.ALL_ALLOWED));
    }

    // Combined Flags //
    function test_hasFlag_SWAP_or_LIQUIDITY() public pure {
        PermissionFlag p = PermissionFlags.SWAP_ALLOWED | PermissionFlags.LIQUIDITY_ALLOWED;
        assertFalse(_has(p, PermissionFlags.NONE));
        assertTrue(_has(p, PermissionFlags.SWAP_ALLOWED));
```

```
        assertTrue(_has(p, PermissionFlags.LIQUIDITY_ALLOWED));

        //@audit-issue => returns true as if it had ALL_ALLOWED permissions
        assertTrue_has(p, PermissionFlags.ALL_ALLOWED));

    }
}
```

Recommended Mitigation: Consider refactoring the `hasFlag` function as follows:

```
    function hasFlag(PermissionFlag permissions, PermissionFlag flag) internal pure returns (bool) {
        if (PermissionFlag.unwrap(flag) == 0) return false;

        return and(permissions, flag) == flag;
    }
}
```

The fix accounts for the case when the provided flag is `NONE`, and also correctly decodes the provided `permissions` to verify the specified `flag` is a subset of the `permissions`.

Securitize: Remove this functionality in commit 0b8d506 as it was not being used.

Cyfrin: Verified.

7.1.4 `PermissionFlags::hasFlag` inconsistent validation for combined flags

Description: `PermissionFlags` allows checking for multiple combinations of flags and verifying whether an account has at least one (OR), or all of them (ALL).

The problem is that `PermissionFlags::hasFlag` doesn't correctly verify the combination of `SWAP_ALLOWED` and `LIQUIDITY_ALLOWED`, it returns false when should return true.

Impact: Using `PermissionFlags::hasFlag` to validate combined flags can lead to incorrect validation, allowing execution when it should not.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import {PermissionFlag, PermissionFlags} from
↪  "../../contracts/uniswap/permissionedPools/libraries/PermissionFlags.sol";

contract PermissionFlagsTest is Test {
    using PermissionFlags for PermissionFlag;

    // Helper to make tests more readable
    function _has(PermissionFlag permissions, PermissionFlag flag) internal pure returns (bool) {
        return permissions.hasFlag(flag);
    }

    function test_hasFlag_SWAP_and_LIQUIDITY() public pure {
        PermissionFlag p = PermissionFlags.SWAP_ALLOWED & PermissionFlags.LIQUIDITY_ALLOWED;
        assertFalse(_has(p, PermissionFlags.NONE));
        assertFalse(_has(p, PermissionFlags.SWAP_ALLOWED));
        assertFalse(_has(p, PermissionFlags.LIQUIDITY_ALLOWED));
        assertFalse(_has(p, PermissionFlags.ALL_ALLOWED));

        assertTrue(p == p);
        //@audit-issue => hasFlag incapable of validating combined flags using AND
        assertTrue(_has(p, p));
    }
```

```
}
```

Recommended Mitigation: Consider restricting the use of `PermissionFlags::hasFlag` to validate individual flags; that is, revert execution when the value of the first parameter (`PermissionFlag permissions`) doesn't match any of the expected individual flags.

Securitize: Remove this functionality in commit 0b8d506 as it was not being used.

Cyfrin: Verified.

7.1.5 `PermissionFlags::and` function will encode the permission for `SWAP_ALLOWED` AND `LIQUIDITY_ALLOWED` as if the account would have `NONE` permissions

Description: Using `PermissionFlags::and` function to compute the combined permission for an account expecting to enforce that the account must have BOTH permissions incorrectly encodes the bits corresponding to the `NONE` permission.

```
`SWAP_ALLOWED` & `LIQUIDITY_ALLOWED` => `NONE`
0x0001 & 0x0002 => 0x0000
```

Recommended Mitigation: Taking into consideration the recommendation to fix issue *`PermissionFlags::hasFlag` incorrectly grants `ALL_PERMISSIONS` to `PermissionFlag::LIQUIDITY_ALLOWED` and `PermissionFlag::SWAP_ALLOWED` * . Consider avoiding `and` when combining flags to assign permissions to an account. Instead, if the wanted result is to verify if an account has both permissions, then do it as follows:

```
    function test_hasFlag_LIQUIDITY_or_SWAP() public pure {
        PermissionFlag p = PermissionFlags.LIQUIDITY_ALLOWED | PermissionFlags.SWAP_ALLOWED;
        // 0x0002          | 0x0001            = 0x0003
        assertTrue(_has(p, p));        // "Does the permission set p contain ALL the bits that are set
        ↪  in p?"
    }
```

So when we call `p.hasFlag(p)`:

- `permissions = p = 0x0003`
- `flag       = p = 0x0003`

Compute:

```
(permissions & flag) == flag
(0x0003 & 0x0003)    == 0x0003
0x0003               == 0x0003   → true!
```

Securitize: Remove this functionality in commit 0b8d506 as it was not being used.

Cyfrin: Verified.

7.1.6 Implement recommended changes to proposed vault whitelister standard

Description: Implement recommended changes to proposed vault whitelister standard.

Securitize: Fixed in commits 28ce1f8, 5ceab6d.

Cyfrin: Verified.

7.2 Gas Optimization

7.2.1 `BaseWhitelister::addOperator, removeOperator` should use `_grantRole` and `_revokeRole` directly and only emit events when they return `true`

Description: `BaseWhitelister::addOperator, removeOperator` use public functions `grantRole, revokeRole`; this is not ideal as these functions:

1) use modifier `onlyRole(getRoleAdmin(role))` but the access control has already been applied via modifier `onlyRole(DEFAULT_ADMIN_ROLE)`
2) call `_grantRole, _revokeRole` which return `bool` - if the returned `bool` is `false, the` ProtocolAuthorized, ProtocolRevoked' event would still be emitted even though no access was granted or revoked

Recommended Mitigation: `BaseWhitelister::addOperator, removeOperator` should directly call `_grantRole, _revokeRole` and only emit the events if the returned `bool` is `true`:

```
function addOperator(address operator) external onlyRole(DEFAULT_ADMIN_ROLE)
↪  notZeroAddress(operator) {
    if(_grantRole(OPERATOR_ROLE, operator)) emit ProtocolAuthorized(operator);
}

function removeOperator(address operator) external onlyRole(DEFAULT_ADMIN_ROLE)
↪  notZeroAddress(operator) {
    if(_revokeRole(OPERATOR_ROLE, operator)) emit ProtocolRevoked(operator);
}
```

Securitize: Fixed in commit c0bd66b.

Cyfrin: Verified.

7.2.2 `Whitelister::whitelist` duplicates checks already performed in `RegistryService::addWallet`

Description: `Whitelister::whitelist` checks that the investor has a valid investor id, and that the given `vaultAddress` is not already a registered wallet.

However the function it calls `RegistryService::addWallet` already performs these checks via the modifiers `investorExists(_id) newWallet(_address)`.

Recommended Mitigation: The modifier checks return the older-style text message errors while the new checks return typed errors. If the textual errors are sufficient, consider removing the duplicate checks. Otherwise if the typed errors are desired the existing code can be kept, it will just have slightly higher gas costs.

Securitize: Acknowledged; the duplicate checks are intentional and were kept for readability and explicitness at the call site.